



PROGRAMMATION ORIENTÉE OBJET

**OBJECTIF: CONCEPTION ET IMPLÉMENTATION EN
JAVA DU SYSTÈME « GESTION DES EMPLOIS DU
TEMPS À L'UASZ »**

Par Ibrahima DIOP

Programmation orientée objet (POO)

INTRODUCTION

Pourquoi la POO

- Pour la **production de programmes** possédant une bonne modularité :
 - ❖ On veut développer une partie d'un programme sans qu'il soit nécessaire de connaître les détails d'implémentation des autres parties
 - ❖ On pourra modifier un module sans que cela n'affecte le reste du programme
- Ainsi faciliter **l'écriture, la maintenance et la réutilisabilité de code.**

Langages de POO

- C++,
- Java,
- C#,
- VB.NET,
- Python,
- PHP (à partir de la version 5),
- Etc.

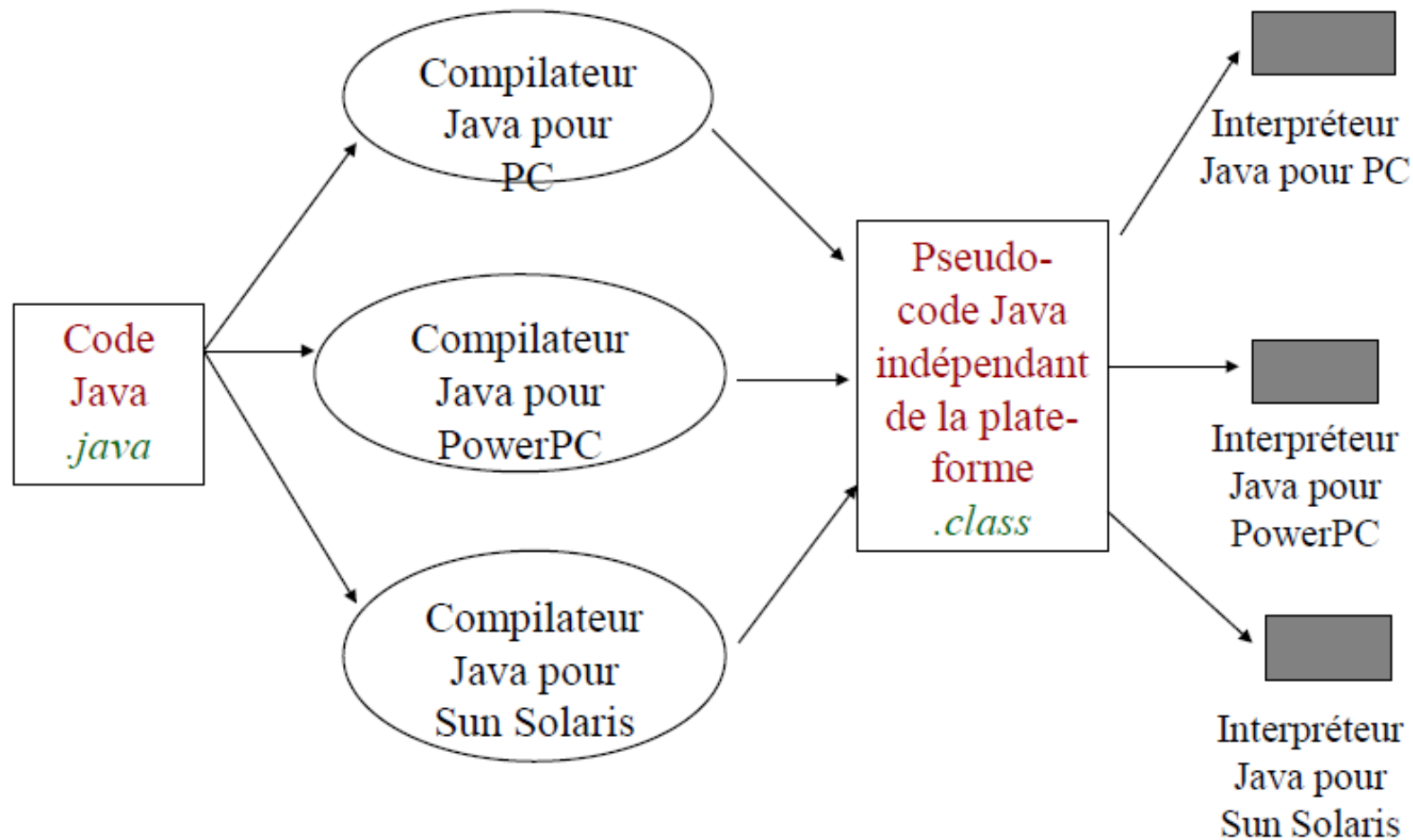
Langages de POO

- C++,
- Java,
- C#,
- VB.NET,
- Python,
- PHP (à partir de la version 5),
- Etc.

C'est quoi Java ?

- Langage orienté objet développé par Sun en 1990.
- Java peut être séparé en deux parties.
 - D'une part, un **programme écrit en langage Java**
 - et d'autre part, une **machine virtuelle (JVM)** qui va se charger de l'exécution de votre programme Java.
- C'est cette plateforme qui garantit la portabilité de Java.

Java est portable



C'est quoi Java ?

- Le Langage Java est dédié à Internet grâce à l'attrait des applets.
- Les **applets** sont des **programmes Java encapsulés dans des pages Web**
 - et qui permettent par exemple l'**animation des pages** ou l'**interactivité entre le serveur et le client Web**.
- Grande richesse des APIs proposées : interfaces graphiques, bases de données, réseau, ...
- Syntaxe proche de C++
- Une grande communauté d'utilisateurs

Kit de développement Java (1)

- **Choix 1** : Le JDK de Sun (oracle.com/java/technologies)
 - Référence pour développer en Java.
 - Comporte tous les outils indispensables à la réalisation et l'exécution d'un programme Java
 - totalement gratuit
 - ne comporte pas de véritable environnement de développement.

Kit de développement Java (2)

- **Choix 2** : Freeware ou shareware offrent une interface graphique au JDK
 - des outils supplémentaires (générateurs d'interface...)
 - Exemple : Eclipse, NetBeans
- **Choix 3** : Environnement de développement professionnel très puissants
 - très chers...
 - IntelliJ
 - *JBuilder 2 de Inprise (anciennement Borland)*
 - *Visual J++ de Microsoft*

Objectif Général

- Vous permettre de concevoir et d'implémenter une solution logicielle modulaire et évolutive avec l'utilisation du paradigme orienté objet.
- **THÈME : CONCEPTION ET IMPLÉMENTATION DU SYSTÈME « GESTION DES EMPLOIS DU TEMPS À L'UASZ »**

Objectifs spécifiques

- **À l'issu de ce cours, vous serez capable:**
 - De modéliser une application selon le concept orienté objet
 - D'écrire les classes identifiées lors de la conception
 - D'intégrer les classes dans un programme exécutable
 - De pouvoir étendre une application existante afin de la faire évoluer (introduction de nouveaux modules, modification de modules existantes, ...)

Pré requis

- Pour suivre ce cours vous devez avoir:
 - Une bonne connaissances de l'algorithmique et de la programmation procédurale

Stratégie pédagogique

- Alternance de **cours théoriques** sous forme d'exposé avec des **Travaux dirigés** (Échanges avec les étudiants) et des **Travaux pratiques**.
- **Ateliers de génie logiciel** pour la conception orientée objet du système de gestion du matériel dans l'UASZ.

Partie 1: Rappel des Concepts et fondamentaux de la POO en JAVA

PLAN

- **Syntaxe de base de JAVA**
 - Les structures de contrôle
 - Tableaux / Types énumérés
 - Les types et les Entrées / Sorties simplifiées
- **Fondements de la POO en JAVA**
 - Classes et objets
 - Package
 - Visibilité des classes, variables et méthodes
 - Héritage & Polymorphisme
 - Classes abstraites et interfaces
- **Utilisation de bibliothèques JAVA**
 - Exceptions
 - Interfaces graphiques avec Javafx
 - Persistance avec JDBC

Partie 1: Rappel des Concepts et fondamentaux de la POO

PLAN

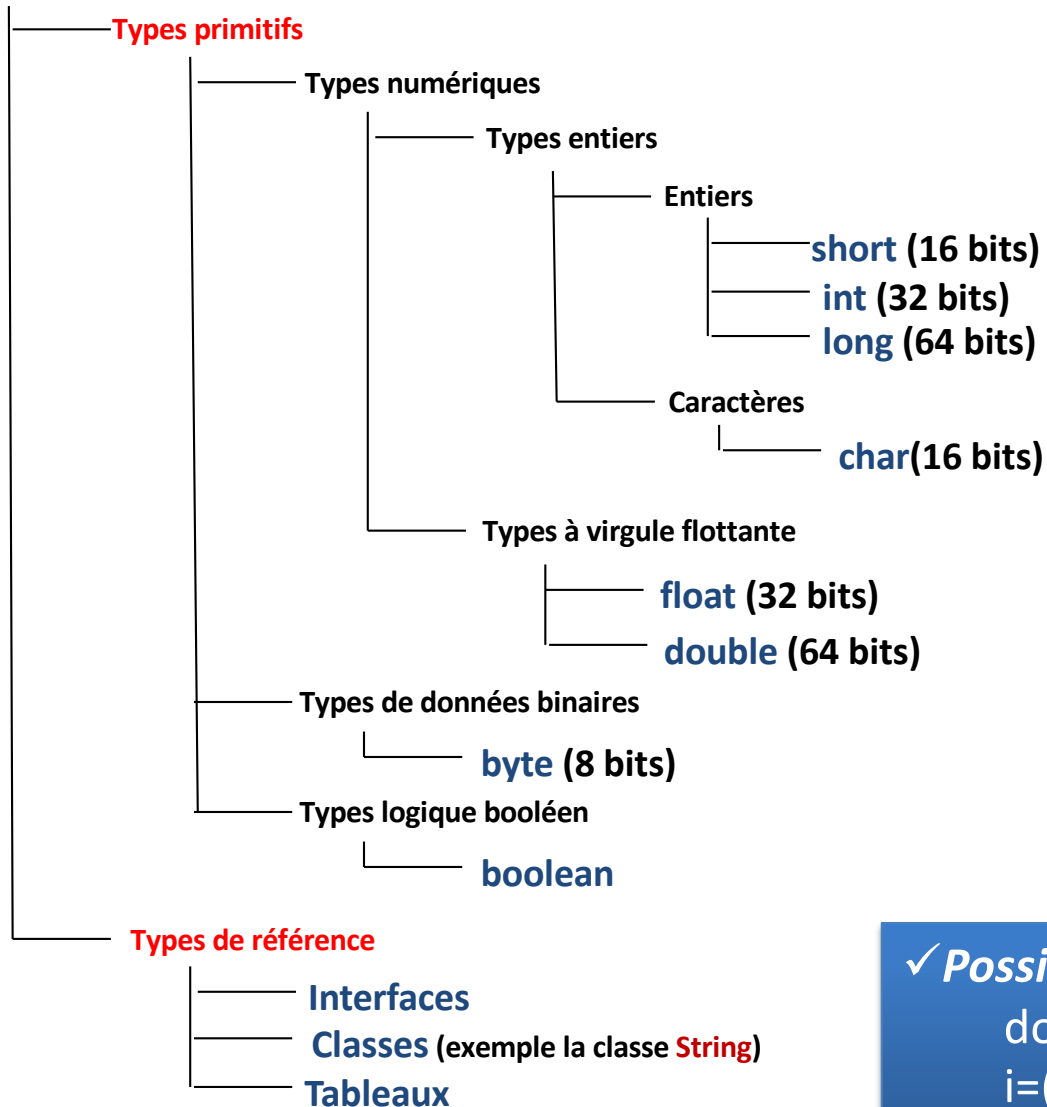
- **Syntaxe de base du langage JAVA**
 - Les structures de contrôle
 - Tableaux / Types énumérés
 - Les types et les Entrées / Sorties simplifiées
- **Fondements de la POO en JAVA**
 - Classes et objets
 - Package
 - Visibilité des classes, variables et méthodes
 - Héritage & Polymorphisme
 - Classes abstraites et interfaces
- **Utilisation de bibliothèques JAVA**
 - Exceptions
 - Interfaces graphiques avec Javafx
 - Persistance avec JDBC

Les commentaires

- Les commentaires tiennent sur une ligne :
`// tout le reste de la ligne est un commentaire`
- Les commentaires sont multi-lignes :
`/* ceci est un commentaire
tenant sur deux lignes
*/`
- Les commentaires sont destinés au système **javadoc** pour la génération d'une documentation API à partir du code :
`/** ceci est un commentaire spécial destiné au
système JavaDoc */`

Les types de données JAVA

Types de données Java



✓ Possibilité de faire du casting comme en C

```
double d; int i;  
i=(int)d; //troncation de d en un entier
```

La classe String

- **String n'est pas un type primitif, c'est une classe**
- **Déclaration de deux String:**

```
String s1, s2;
```

- **Initialisation :**

```
s1 = 'Hello';
```

```
s2 = 'le monde';
```

- **Déclaration et initialisation :**

```
String s3 = 'Hello';
```

- **Concaténation :**

```
String s4 = s1 + ' ' + s2;
```

Programmation orientée objet (POO)

Syntaxe de base du langage JAVA

LES STRUCTURES DE CONTRÔLE

Instructions conditionnelles (IF)

Syntaxe:

```
if (condition)
{
instruction1
}
else
{
instruction2
}
```

Exemple:

```
If ( x > 0)
{
System.out.println('positif ');
    x-- ;
}
else
System.out.print('négatif ');
```

Instructions itératives (Pour...Faire)

Syntaxe:

```
for (initialisation; condition; expression)
{
    instructions
}
```

Exemple:

```
for (int i=0 ; i<5 ; i++)
    System.out.print('Salut!');
```

Instructions itératives (TantQue...Faire...)

Syntaxe:

```
while (condition)
{
    instructions
}
```

Exemple:

```
int i = 10 ;
while ( i>0)
{
    System.out.println(i);
    i-- ;
}
```

Instructions itératives (Faire...TantQue...)

Syntaxe:

```
do
{
  instructions
}
while (condition);
```

Exemple:

```
do
  a++
while (a != b);
```


L'instruction Switch

Syntaxe:

```
switch (expression)
{
    case valeur1:
        instructions; break;
    case valeur2:
        instructions; break;
    ...
    default:
        instructions
}
```

Exemple:

```
switch (i) {
    case 10 : System.out.println('x
        égale 10 '); break ;
    case 100 : System.out.println('x
        égale 100 '); break ;
    default : System.out.println('x est
        différent de 10 et de 100 ');
}
```

Programmation orientée objet (POO)

Syntaxe de base du langage JAVA

TABLEAUX / TYPES ÉNUMÉRÉS

Déclaration des tableaux

- Les tableaux se comportent un peu comme des objets.
- Deux syntaxes pour la déclaration:
- **TypeDesElements**[] **NomDuTableau**;
- **TypeDesElements** **NomDuTableau**[];
- *Exemple:*
 - `int[ ] tab_entiers; ou int tab_entiers[ ];`
 - `String[ ] tab_chaines;`
- Le type des éléments du tableau peut être une classe existante.

Instanciation d'un tableau

- *InstanciationTypeDesElements[]*
NomDuTableau = new
TypeDesElements[TailleDuTableau];
- *Exemple*
 - int[] tab_entiers = new int[15];
 - String[] chaines = new String[3];

Exemple de saisie d'un tableau

```
public static void main(String[] args) {  
    int[] t = new int[5];  
    for (int i=0; i<5; i++){  
        t[i] = Integer.parseInt(args[i]);  
    }  
}
```

Matrices

- TypeDesElements[][] NomDuTableau;
- TypeDesElements NomDuTableau[][];
- *Exemple*
 - int[][] matrice
 - *ou int matrice[][];*

Déclaration d'un type énuméré (1/2)

- qualifieurs enum identif { identif, identif, ... identif }
- *Exemples :*
 - public enum PointCardinal { NORD, SUD, EST, OUEST }
 - public enum JourSemaine { LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE }

Déclaration d'un type énuméré (2/2)

- Ces déclarations créent 2 nouveaux types publics **PointCardinal** et **JourSemaine**.
- Elles doivent être écrites dans 2 fichiers séparés, respectivement nommés *PointCardinal.java* et *JourSemaine.java*.

Programmation orientée objet (POO)

Syntaxe de base du langage JAVA

TYPES ET ENTRÉES / SORTIES

La méthode printf

- La classe **java.io.PrintStream** contient une méthode **printf** qui fonctionne presque de la même façon qu'en C.
- **Exemple** :
 - `int degre = 28;`
 - `System.out.printf('Il fait %d degres\n', degre);`
 - `// %d` indicateur de format pour des entiers
- Comme en C on utilise des **indicateurs de format** commençant par le symbole **%**.

javax.swing.JOptionPane

- String **entree** =
JOptionPane.showInputDialog("demande de données d'entrées");
- int **sortie1**=Integer.parseInt(**entree**);
- ...
- int **sortie2** = 2***sortie1**;
- ...
- **JOptionPane.showMessageDialog**(null, "la sortie 1 est "+**sortie1**);
- **System.out.println**("la sortie 2 est "+**sortie2**);

La classe java.util.Scanner

```
Scanner entree = new Scanner(System.in);  
System.out.println("Donner votre prenom et votre nom:");  
String prenom = entree.next(); String nom = entree.next();  
System.out.println("Donner votre age:");  
int age = entree.nextInt();  
entree.nextLine();  
System.out.println("Donner une phrase:");  
String phrase = entree.nextLine();  
System.out.printf("%s %s, %d ans, dit %s\n", prenom, nom, age,  
phrase);
```

La classe `java.util.Scanner`

Offre des méthodes pour **lire** un **flux de données** formatées :

- **`int nextInt()`** : lecture de la prochaine valeur de type `int`.
- **`long nextLong()`** : lecture de la prochaine valeur de type `long`.
- **`float nextFloat()`** : lecture de la prochaine valeur de type `float`.
- **`double nextDouble()`** : lecture de la prochaine valeur de type `double`.
- **`String nextLine()`** : lecture de tous les caractères se présentant dans le flux d'entrée jusqu'à une marque de fin de ligne et renvoi de la chaîne ainsi construite. **La marque de fin de ligne est consommée, mais n'est pas incorporée à la chaîne produite.**

La classe java.util.Scanner

Offre aussi toute une série de **méthodes booléennes** pour tester le type de la prochaine donnée disponible, sans lire cette dernière :

- ✓ **boolean hasNext()** : Y a-t-il une donnée disponible pour la lecture
- ✓ **boolean hasNextLine()** : Y a-t-il une ligne disponible pour la lecture
- ✓ **boolean hasNextInt()** : La prochaine donnée à lire est-elle de type int
- ✓ **boolean hasNextLong()** : La prochaine donnée à lire est-elle de type long
- ✓ **boolean hasNextFloat()** : La prochaine donnée à lire est-elle de type float
- ✓ **boolean hasNextDouble()** : La prochaine donnée à lire est-elle de type double

Partie 1: Rappel des Concepts et fondamentaux de la POO

PLAN

- Syntaxe de base du langage JAVA
 - Les structures de contrôle
 - Tableaux / Types énumérés
 - Les types et les Entrées / Sorties simplifiées
- **Fondements de la POO en JAVA**
 - **Classes et objets**
 - **Package**
 - **Visibilité des classes, variables et méthodes**
 - **Héritage & Polymorphisme**
 - **Classes abstraites et interfaces**
- **Utilisation de bibliothèques JAVA**
 - **Exceptions**
 - **Interfaces graphiques avec Javafx**
 - **Persistance avec JDBC**

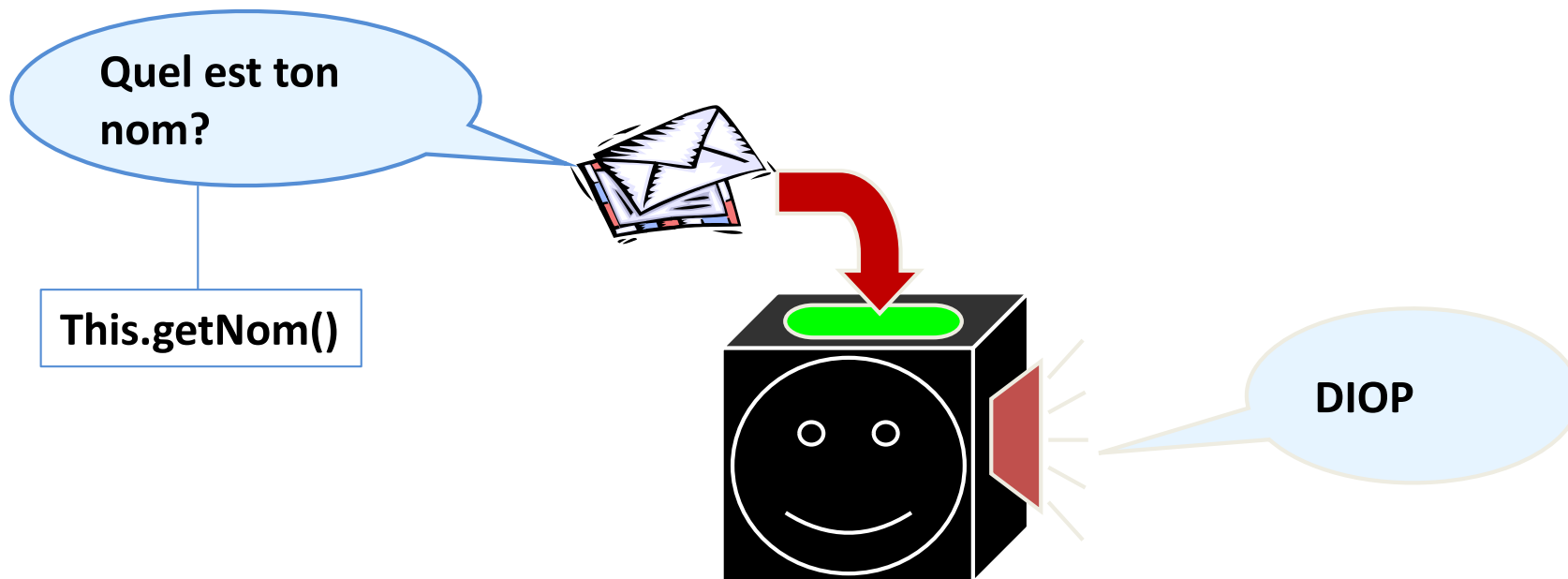
Programmation orientée objet (POO)

Fondements de la POO en JAVA

CLASSES ET OBJETS

Qu'est-ce qu'un objet?

- Un objet est une boîte noire qui **contient des données**, **reçoit** et **envoie** des messages



Que contient la boîte noire ?

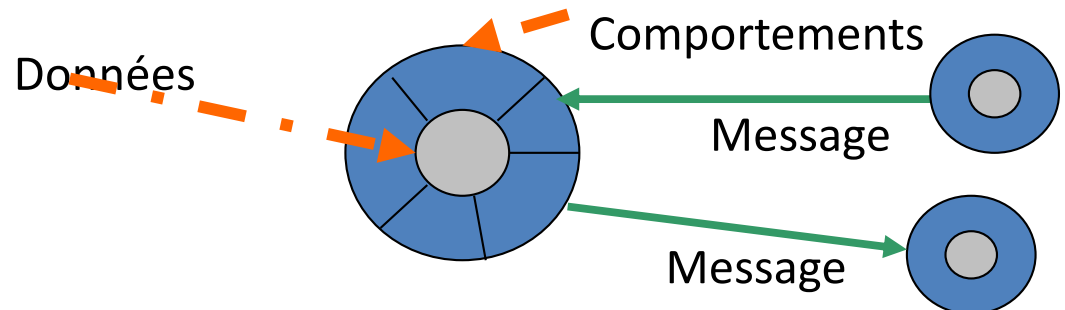
- Du code → Traitements composés d'instructions → **Comportements**
- Des données → L'information traitée par les instructions et qui caractérise l'objet → **Etats** ou **Attributs**
 - Données et traitements sont donc indissociables
 - Exemple : Chien1.dessin(taille)
 - Exemple : dessin(taille) (En procédural)

Pourquoi une boîte noire?

- L'utilisateur d'un objet ne devrait jamais avoir à plonger à l'intérieur de la boîte
 - Les messages définissent l'interface des objets donc la façon d'interagir avec eux
 - Il faut et il suffit de **connaître l'interface des messages** pour pouvoir exploiter l'objet à 100%, sans jamais avoir à connaître le contenu exact de la boîte ni les traitements qui l'animent

Pourquoi une boîte noire?

- L'intégrité de la boîte et la sécurité des données peut être assurée
- Les utilisateurs d'un objet ne sont **pas menacés** si le concepteur de l'objet en change les détails ou la mécanique interne
- **ENCAPSULATION**



Comment les objets sont-ils définis?

- Par leur classe, qui détermine toutes leurs caractéristiques
 - (les attributs et les comportements possibles)
- La classe **détermine** tout ce que **peut contenir** un objet et tout ce qu'on **peut faire** de cet objet
- Classe = Structure d'un d'objet, Moule, Définition, Modèle
- Objet = Instance d'une classe
- Une classe peut-être composite $\Leftrightarrow$ peut contenir elle-même des objets d'autres classes
 - Ex: Une **voiture** contient un **moteur** qui contient des cylindres...
 - Ex: Un **chien** possède des **pattes**...



Quelques exemples?

Classes	Attributs ou États	Comportements
Etudiant	N° carte, nom, prénom, date de naissance, nom de classe	inscrire, chercher son emploi du temps, recevoir e-mail, ...
Salle	N°, type, nombre de places, N° bâtiment	Être occuper, être libre, ...



Classes	Objets ou instances de classes
Etudiant	C00211, Niang, Mamadou, 10/12/1995, Master 1 INFORMATIQUE CD0234, Diop, Abdoulaye, 07/02/1992, Master 2 INFORMATIQUE
Salle	S01, Salle de Cours, 120, Bâtiment F S13, Salle de TP, 25, Bâtiment F

Classes et objets - Résumé



Quel est ton nom ?

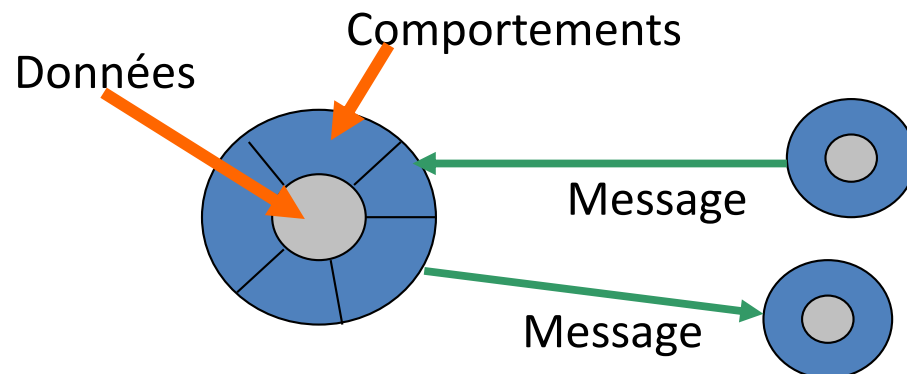
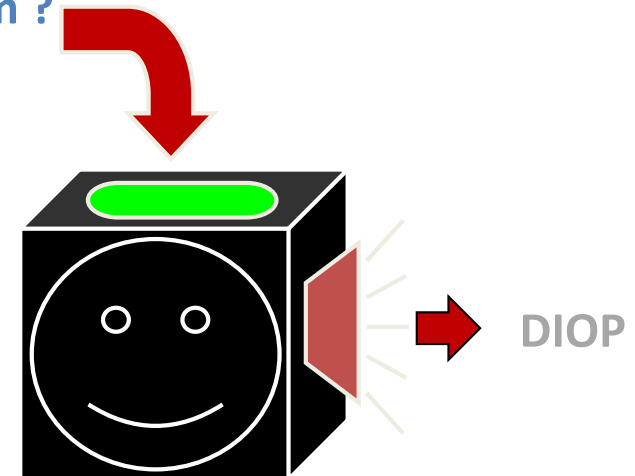
Classe => (1) **Moule**, (2) **Définition**, (3) **Structure**, (4) **Modèle**

Objet => (1) **instance d'une classe** (brique)

Objet => (2) **boite noire**

Objet => (3) contient **données encapsulées** et **comportements**

Objet => (4) communication **par messages**



nom = **DIOP**

prenom = **Abdoulaye**

getNom() => **DIOP**

getPrenom => **Abdoulaye**

setNom('GOUDIABY') => nom = **GOUDIABY**

setPrenom ('Abdou') => prenom = **Abdou**

Les identificateurs

Mêmes règles de formation des identificateurs qu'en C.

- **Recommandations de SUN** : Java Code Conventions (<http://java.sun.com/docs/codeconv>)
- ✓ Le **nom d'une classe ou d'une interface** est fait d'un mot ou de la concaténation de plusieurs mots.
Chacun commence par une majuscule : **Etudiant**, **EtudiantBoursier**, etc.
- ✓ Les **noms des méthodes et des variables** commencent par une minuscule.
Lorsqu'ils sont formés d'une concaténation de mots, chacun, sauf le premier, commence par une majuscule : **afficher**, **toString**, **nom**, **numeroInscription**, etc.
- ✓ Les **constantes de classe** (variables **static final**) sont écrites entièrement en majuscules : **Integer.MAX_VALUE**

Définition d'une classe

```
class NomDeLaClasse {
```

Déclarations des variables

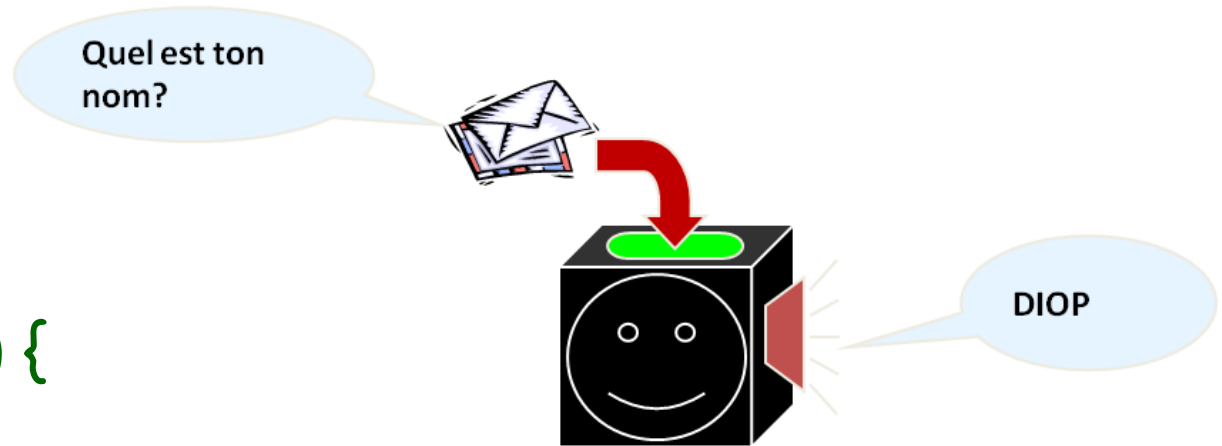
Déclarations des méthodes

- *Déclaration des constructeurs*
- *Déclaration des getteurs et des setteurs*
- *Déclaration des autres méthodes*

```
}
```

Exemple de définition de classe

```
class Etudiant{  
    // variables  
    int num ;  
    String nom ;  
    String prenom ;  
  
    // méthodes  
    String getNom () {  
        return nom;  
    }  
    ...  
}
```



Utilisation d'une classe

```
class EtudiantUtil {  
    public static void main (String[] args) {  
        Etudiant f = new Etudiant();  
        f.num = 8; f.nom = 'Sall'; f.prenom = 'Amadou';  
        System.out.println(f.getNom());  
    }  
}
```

- Instanciation avec l'opérateur *new*
- Java s'occupe de la destruction des objets non utilisés.
→ pas besoin de 'destructeurs'.

Le désignateur (auto-référence) **this**

- **this** est une variable qui permet :
 - ✓ de faire référence à l'objet de la classe courante (celle qu'on est en train de définir).
 - ✓ ou de désigner ses attributs ou ses méthodes.
- **this** est généralement utilisé pour lever l'ambiguïté sur un identificateur lorsque le paramètre d'une méthode porte le nom de l'attribut de l'objet qu'il est chargé de modifier.
- *Exemple :*
- ```
public void set(int num, String nom, String prenom) {
 this.num = num; this.nom = nom; this.prenom = prenom ;
}
```

# Constructeurs spécifiques

```
public class Etudiant {
 public int num;
 public String nom, prenom ;
}
```

```
public Etudiant(int n) {
 this.num=n;
}
```

```
public Etudiant(int n, String defaultNom, String defaultPrenom) {
 this.num=n;
 this.nom= defaultNom;
 this.prenom=defaultPrenom;
}
```

NB: Les divers constructeurs doivent avoir des signatures différentes (*nombre d'arguments ou types des arguments différents*).

# Utilisation d'une classe

```
public class EtudiantUtil {
 public static void main (String[] args) {
 Etudiant f = new Etudiant();
 f.num = 8; f.nom = 'Sall'; f.prenom = 'Amadou';
 System.out.println(f.getNom());
 }
}
```

```
public class EtudiantUtil {
 public static void main (String[] args) {
 Etudiant f = new Etudiant(8,'Sall','Amadou');
 System.out.println(f.getNom());
 }
}
```

# La méthode main et les arguments de la ligne de commande (1)

- Premier sous-programme exécuté au lancement d'une application Java.

- Signature:

```
public static void main(String[] args){
```

```
 ...
```

```
}
```

- Le paramètre est un tableau de chaînes utilisé pour les arguments transmis à partir de la ligne de commande

# La méthode main et les arguments de la ligne de commande (2)

```
public class EtudiantUtil {
 public static void main (String[] args) {
 int num = Integer.parseInt(args[0]);
 String nom=args[1];
 String prenom=args[2];
 Etudiant f = new Etudiant(num,nom,prenom);
 System.out.println('Le nom de l'étudiant est : '+f.getNom());
 }
}
```

*Utilisation:*

- *java EtudiantUtil* **12 Abdoulaye DIOP**
- ==> **Le nom de l'étudiant est : DIOP**



Programmation orientée objet (POO)

Fondements de la POO en JAVA

# **PACKAGES**

# Définition et Utilité

- Un package est un ensemble de classes et d'autres packages regroupés sous un nom.
- C'est l'adaptation du concept de librairie ou de bibliothèque.
- Servent à structurer l'ensemble des classes et interfaces.
- Augmentent la lisibilité des applications en structurant l'ensemble des classes selon une arborescence.
- Facilitent la recherche de l'emplacement physique des classes.
- Empêchent la confusion entre des classes de même nom.

# Généralités

- Lors de la définition d'un fichier source Java, on peut préciser son appartenance à un package en utilisant le mot-clé **package**:
  - **package** mon\_package;
- Toutes les classes définies dans ce répertoire font ainsi partie du package mon\_package.
- Exemple : **package** formesGeo;
  - Si la classe **Carre** se trouve dans un package formesGeo, son nom complet est formesGeo.Carre;
  - Le fichier **Carre.class** doit être placé dans un répertoire nommé formesGeo.

# Importation

- Pour pouvoir utiliser une classe d'un package dans un autre fichier, il faut l'importer en faisant :  
`import nom_du_package.nom_classe;`  
– `import formesGeo.Carre;`
- On peut importer toutes les classes d'un package en même temps:  
– `import nom_du_package.*; import java.io.*;`
- Le package `java.lang` qui gère les types de données et les éléments de base du langage est automatiquement importé.

# ***Les packages prédéfinis (1/2)***

- Les API Java sont organisées en deux parties:
  - **JavaCoreAPI** : API de base implantée dans tout interpréteur Java. Ces classes appartiennent au package “java”.
  - **JavaStandardExtensionAPI** : extensions normalisées par Sun au travers d'interfaces et implantées par les éditeurs.
- Ces classes appartiennent aux packages “javax” et “org”.
- Documentation sur toutes les classes Java disponible sur le site officiel de Java : <http://java.sun.com/>

# *Les packages prédéfinis*

- Les classes sont regroupées par thèmes dans des packages.
- Exemples:
  - **java.lang**: types de données et éléments de base du langage (classesObject, String, Boolean, Integer, Float, Math, System (fonctionssystème), Runtime(mémoire,processus), etc.)
  - **java.util**: structures de données et utilitaires (Dates, Collections, etc.)
  - **java.io**: bibliothèque des entrées / sorties, fichiers.
  - **java.net**: bibliothèque réseau
  - **java.awt**: librairie graphique
  - **java.applet**: librairie des applets.
  - **java.security**: sécurité (contrôle d'accès, etc.)
  - **java.rmi**: applications distribuées.
  - **java.sql**: accès aux BD (JDBC)

Programmation orientée objet (POO)

Fondements de la POO en JAVA

# **VISIBILITÉ DES CLASSES, VARIABLES ET MÉTHODES**

# Portée des autorisations

- La portée de cette autorisation dépend de l'élément à laquelle elle s'applique

| Élément  | Autorisations                                                                       |
|----------|-------------------------------------------------------------------------------------|
| Variable | Lecture et écriture                                                                 |
| Méthode  | Appel de la méthode                                                                 |
| Classe   | Instanciation d'objets de cette classe et accès aux variables et méthodes de classe |



# Visibilité des variables et méthodes

- Le principe d'encapsulation permet d'introduire celui de protection des attributs et méthodes.
- **Les Modificateurs de visibilité en Java :**
  - **public** : accessible par toutes les classes. Hérité par les sous classes.
  - **private** : accessible que par les seules méthodes de sa classe. Non hérité.
  - **protected** : accessible par les classes du même package. Hérité par les sous classes.
  - **par défaut** : accessible par les classes du même package. Hérité par les sous classes que si elles se trouvent dans le même package.

# Exemples de Visibilités

```
public class Etudiant {
 public String nom;
 private String numIns;

 public Etudiant(String nom) {
 this.nom = nom;
 }

 // methodes accesseurs
 public String getNumIns() { return this.numIns; }
 public void setNumIns(String numIns) {
 if (numIns==null || numIns.length()!=11) return;
 this.numIns = numIns;
 }

 public void afficher(){
 System.out.println('etudiant :'+ nom);
 }
}
```

Programmation orientée objet (POO)

# **HÉRITAGE & POLYMORPHISME**